Descripción del Software - Laboratorio de Bases de Datos

Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Laboratorio de Bases de Datos
Titulación: Grado en Ingeniería Telemática
Universidad: Universidad Rey Juan Carlos (URJC)
Categoría: Programas de Ordenador
SWHID: https://archive.softwareheritage.org/browse/origin/directory/?origin_url=https://github.com/dlumb
11-03T18:36:42.505000%2B00:00
SWHID ID swh:1:dir:61639b22e4ef1e9dee2adabb7738c6e9f02c0e93
Licencia: "Atribución-CompartirIgual 4.0 Internacional" de Creative Commons.
Autores: David Moreno Lumbreras y Daniela Patricia Feversani

Índice

1. Resumen General
2. Laboratorio 0 (L0) - Configuración del Entorno
3. Laboratorio 1 (L1) - Consultas SQL Básicas
4. Laboratorio 2 (L2) - JOINs y Subconsultas
5. Laboratorio 3 (L3) - Acceso Programático con Python
6. Requisitos del Sistema
7. Instrucciones de Uso

Resumen General

Este conjunto de materiales proporciona una colección completa de recursos para el aprendizaje práctico de bases de datos relacionales utilizando PostgreSQL. El material está organizado en cuatro laboratorios progresivos que abarcan desde la configuración inicial del entorno hasta el acceso programático mediante Python.

Bases de Datos Utilizadas

El material trabaja con tres bases de datos de ejemplo:

1. **Universidad**: Sistema académico con estudiantes, profesores, asignaturas y matrículas.
2. **bpsimple**: Sistema de ventas minorista con clientes, productos, pedidos e inventario.
3. **DVDRental**: Sistema completo de alquiler de películas (base de datos de muestra de PostgreSQL).

Laboratorio 0 (L0) - Configuración del Entorno

Descripción

Configuración inicial del entorno de desarrollo con PostgreSQL y herramientas de administración mediante contenedores Docker.

Contenido

Carpeta `postgresql/` **Archivo**: `docker-compose.yml`

Descripción: Archivo de configuración Docker Compose que despliega un entorno completo de PostgreSQL con pgAdmin4.

Servicios incluidos: - **PostgreSQL 16**: Motor de base de datos relacional - Puerto: 5432 - Usuario por defecto: postgres - Contraseña: changeme (configurable) - Volumen persistente para datos

- **pgAdmin 4**: Interfaz web de administración
  - Puerto: 5050
  - Credenciales: pgadmin4@pgadmin.org / admin (configurables)
  - Interfaz de administración gráfica para PostgreSQL

Uso:

```
cd postgresql/
docker-compose up -d
```

Acceso: - PostgreSQL: `localhost:5432` - pgAdmin: `http://localhost:5050`

Carpeta `SQL/` Scripts SQL para la creación e inicialización de las bases de datos.

Archivos:

1. **`BDEjemploUni_Create.sql`**
   - Creación del esquema de la base de datos Universidad
   - Define tablas: Estudiantes, Profesores, Asignaturas, Matriculas
   - Establece claves primarias y foráneas
   - Restricciones de integridad referencial
2. **`BDEjemploUni_Insert.sql`**
   - Datos de prueba para la base de datos Universidad
   - Población de todas las tablas con datos realistas
   - Incluye estudiantes, profesores, asignaturas y matrículas
3. **`create_tables-bpsimple.sql`**
   - Esquema de la base de datos bpsimple (versión básica)
   - Tablas: customer, item, orderinfo, orderline, barcode, stock
4. **`create_tables-bpsimple-relaciones.sql`**

    - Esquema completo de bpsimple con todas las relaciones
    - Incluye claves foráneas y restricciones de integridad
5. **`pop-all-tables.sql`**
    - Datos de prueba para bpsimple (versión básica)
6. **`pop-all-tables-relaciones.sql`**
    - Datos de prueba para bpsimple con relaciones completas
7. **`drop_tables.sql`**
    - Script de limpieza para eliminar todas las tablas
    - Útil para reiniciar el entorno

Orden de ejecución recomendado:

```
-- Para Universidad:
\i BDEjemploUni_Create.sql
\i BDEjemploUni_Insert.sql

-- Para bpsimple:
\i create_tables-bpsimple-relaciones.sql
\i pop-all-tables-relaciones.sql
```

Laboratorio 1 (L1) - Consultas SQL Básicas

Descripción

Introducción al lenguaje SQL mediante consultas básicas de selección, filtrado, ordenación y funciones de agregación.

Contenido

`bpsimple.sql` (300 líneas) Colección de consultas SQL básicas e intermedias sobre la base de datos bpsimple.

Temas cubiertos:

- SELECT básico y proyección de columnas
- Cláusula WHERE y operadores de comparación
- Operadores LIKE para búsqueda de patrones
- Operadores IN, BETWEEN, NOT IN
- ORDER BY y ordenación múltiple
- LIMIT para limitar resultados
- Funciones de agregación: COUNT, SUM, AVG, MAX, MIN
- GROUP BY y HAVING para agrupamiento
- Funciones de texto: UPPER, LOWER, CONCAT, SUBSTRING, TRIM
- Funciones de fecha: EXTRACT, TO_CHAR, AGE
- Operaciones matemáticas: ROUND, SQRT, POWER, ABS
- Expresiones CASE para lógica condicional
- Función WIDTH_BUCKET para clasificación

Ejemplos de consultas:

```sql
-- Filtrado básico
SELECT description, sell_price FROM item WHERE sell_price > 10;

-- Agregación con GROUP BY
SELECT town, COUNT(*) FROM customer GROUP BY town;

-- Funciones de fecha
SELECT EXTRACT(YEAR FROM date_placed) AS año FROM orderinfo;

-- Expresiones CASE
SELECT description,
  CASE
    WHEN sell_price < 5 THEN 'Económico'
    WHEN sell_price < 15 THEN 'Medio'
    ELSE 'Caro'
  END AS categoria
FROM item;
```

`universidad.sql` (300+ líneas) Consultas SQL básicas adaptadas al dominio académico.

Temas similares a bpsimple, aplicados a: - Gestión de estudiantes y matrículas - Análisis de notas y promedios - Consultas sobre profesores y departamentos - Estadísticas de créditos y asignaturas

---

Laboratorio 2 (L2) - JOINs y Subconsultas

Descripción

Consultas avanzadas que combinan múltiples tablas mediante diferentes tipos de JOINs y utilizan subconsultas para resolver problemas complejos.

Contenido

`bpsimple.sql` (487 líneas) Consultas avanzadas con JOINs y subconsultas sobre el sistema de ventas.

Temas cubiertos:

- INNER JOIN para combinar tablas relacionadas
- LEFT JOIN / RIGHT JOIN para incluir registros sin coincidencias
- Múltiples JOINs en una consulta (3-4 tablas)
- Subconsultas en WHERE
- Subconsultas correlacionadas
- Operadores IN, NOT IN, EXISTS, NOT EXISTS

- Agregaciones con JOINs
- HAVING con condiciones complejas
- COALESCE para manejar valores NULL
- Window functions: RANK, DENSE_RANK, ROW_NUMBER
- Funciones de ventana con PARTITION BY

Ejemplos de consultas:

```sql
-- JOIN de múltiples tablas
SELECT c.fname, c.lname, i.description, ol.quantity
FROM customer c
  INNER JOIN orderinfo oi ON c.customer_id = oi.customer_id
  INNER JOIN orderline ol ON oi.orderinfo_id = ol.orderinfo_id
  INNER JOIN item i ON ol.item_id = i.item_id;

-- Subconsulta en WHERE
SELECT description, sell_price
FROM item
WHERE sell_price > (SELECT AVG(sell_price) FROM item);

-- Window function
SELECT description, sell_price,
  RANK() OVER (ORDER BY sell_price DESC) AS ranking
FROM item;
```

`universidad.sql` (500+ líneas) Consultas avanzadas para análisis académico.

Casos de uso: - Análisis de rendimiento estudiantil por departamento - Rankings de estudiantes por nota media - Estadísticas de profesores y carga docente - Identificación de patrones de matriculación - Análisis comparativo entre cursos y semestres

`dvdrental.sql` (600+ líneas) Consultas sobre la base de datos DVDRental (la más completa).

Esquema DVDRental: - **16 tablas**: actor, film, customer, rental, payment, inventory, store, etc. - **Miles de registros** de películas, actores, clientes y alquileres - Datos realistas para análisis complejo

Temas adicionales: - Análisis temporal de alquileres - Cálculo de rentabilidad por película/categoría - Identificación de mejores clientes - Análisis geográfico (países, ciudades) - Patrones de colaboración entre actores - Optimización de inventario

Ejemplos de análisis:

```sql
-- Top 5 películas más rentables
SELECT f.title, SUM(p.amount) AS ingresos
```

```sql
FROM film f
  JOIN inventory i ON f.film_id = i.film_id
  JOIN rental r ON i.inventory_id = r.inventory_id
  JOIN payment p ON r.rental_id = p.rental_id
GROUP BY f.film_id, f.title
ORDER BY ingresos DESC
LIMIT 5;

-- Actores que han trabajado juntos
SELECT a1.first_name, a1.last_name,
       a2.first_name, a2.last_name,
       COUNT(*) AS peliculas_juntos
FROM actor a1
  JOIN film_actor fa1 ON a1.actor_id = fa1.actor_id
  JOIN film_actor fa2 ON fa1.film_id = fa2.film_id
  JOIN actor a2 ON fa2.actor_id = a2.actor_id
WHERE a1.actor_id < a2.actor_id
GROUP BY a1.actor_id, a2.actor_id
HAVING COUNT(*) > 3;
```

Laboratorio 3 (L3) - Acceso Programático con Python

Descripción

Integración de bases de datos PostgreSQL con aplicaciones Python utilizando diferentes librerías y enfoques (síncrono, asíncrono, ORM).

Contenido

Archivos de Configuración `myconfig.ini`

```ini
[DEFAULT]
host = localhost
port = 5432
dbname = universidad
user = postgres
password = changeme
```

Archivo de configuración centralizado para credenciales de conexión.

Scripts de Conexión **1. `universidad_01_connection.py`** - **Librería**: psycopg3 (oficial de PostgreSQL) - **Propósito**: Conexión básica a PostgreSQL - **Características**: - Uso de context managers (`with`) - Lectura de configuración desde archivo INI - Manejo de excepciones - Cierre apropiado de conexiones

Uso:

```
python universidad_01_connection.py
```

Consultas y Recuperación de Datos **2. `select_top_n.py`** - **Descripción**: Consultas parametrizadas con psycopg3 - **Características**: - Ejecución de SELECT con parámetros - Iteración sobre resultados - Impresión formateada de datos

3. `sa_core_select.py` - **Librería**: SQLAlchemy Core - **Descripción**: Consultas SQL usando SQLAlchemy sin ORM - **Características**: - Construcción de DSN (Data Source Name) - Uso de `text()` para SQL literal - Parámetros nombrados (`:n`) - Acceso a resultados como mappings (diccionarios)

Ejemplo:

```
res = conn.execute(text("""
    SELECT matricula, nombre FROM estudiantes
    WHERE edad > :edad
"""), {"edad": 20})
```

4. `asyncpg_select.py` - **Librería**: asyncpg (driver asíncrono de alto rendimiento) - **Descripción**: Consultas asíncronas a PostgreSQL - **Características**: - Programación asíncrona con `async/await` - Mayor rendimiento para aplicaciones concurrentes - Sintaxis de parámetros posicionales (`$1`, `$2`) - Resultados como `asyncpg.Record`

Ejemplo:

```
async def consultar():
    conn = await asyncpg.connect(...)
    rows = await conn.fetch(
        "SELECT * FROM estudiantes LIMIT $1", 5
    )
```

Análisis de Datos con Pandas **5. `pandas_read_sql.py`** - **Librerías**: pandas + SQLAlchemy - **Descripción**: Lectura directa de SQL a DataFrames - **Características**: - Conversión automática de resultados a DataFrame - Integración con el ecosistema de análisis de datos - Fácil manipulación y visualización posterior

Ejemplo:

```
df = pd.read_sql(
    "SELECT * FROM estudiantes ORDER BY edad",
    engine
```

```
)
print(df.head())
```

6. `pandas_grafico.py` - Descripción: Visualización de datos con matplotlib - **Características**: - Generación de gráficos desde consultas SQL - Análisis visual de tendencias - Exportación de visualizaciones

---

Exportación de Datos **7. `to_csv.py` - Descripción**: Exportación de resultados a archivos CSV - **Características**: - Extracción de datos desde PostgreSQL - Generación de archivos CSV - Procesamiento por lotes

Salida: `matriculas_export.csv` - Archivo CSV de ejemplo con datos exportados - Formato compatible con Excel y otras herramientas

---

MongoDB (Bases de Datos NoSQL) **`mongodb/docker-compose.yml` - Servicios**: - **MongoDB**: Base de datos NoSQL orientada a documentos - Puerto: 27017 - Usuario: admin / password - Volumen persistente

- **Mongo Express**: Interfaz web de administración
  - Puerto: 8081
  - Navegador web para explorar colecciones

Propósito: Introducción a bases de datos NoSQL como comparación con PostgreSQL.

Uso:

```
cd mongodb/
docker-compose up -d
```

Acceso: `http://localhost:8081`

Requisitos del Sistema

Software Base

- **Docker Desktop** 20.10 o superior
- **Docker Compose** 1.29 o superior
- **Python** 3.8 o superior (para L3)
- **PostgreSQL Client** (psql) - opcional para ejecución manual de scripts

Librerías Python (L3)

```
pip install psycopg[binary]
pip install sqlalchemy
```

```
pip install pandas
pip install asyncpg
pip install matplotlib
```

O mediante archivo `requirements.txt`:

```
psycopg[binary]>=3.0
sqlalchemy>=2.0
pandas>=1.5
asyncpg>=0.27
matplotlib>=3.5
```

Recursos del Sistema

- **RAM**: Mínimo 4 GB (recomendado 8 GB)
- **Disco**: 2 GB libres para contenedores e imágenes
- **Procesador**: Cualquier arquitectura compatible con Docker (x86_64, ARM64)

Instrucciones de Uso

Paso 1: Configuración Inicial (L0)

```
# 1. Clonar o descargar el material
cd L0/postgresql

# 2. Iniciar contenedores
docker-compose up -d

# 3. Verificar que los servicios están activos
docker ps

# 4. Acceder a pgAdmin
# Navegador: http://localhost:5050
# Email: pgadmin4@pgadmin.org
# Password: admin

# 5. Conectar pgAdmin a PostgreSQL
# Host: postgres_container
# Port: 5432
# User: postgres
# Password: changeme
```

Paso 2: Creación de Bases de Datos

Opción A: Usando pgAdmin 1. Abrir pgAdmin en el navegador 2. Crear nueva base de datos (ej: "universidad") 3. Abrir Query Tool 4. Copiar y ejecutar scripts SQL de L0/SQL/

Opción B: Usando psql (línea de comandos)

```
# Conectar al contenedor
docker exec -it postgres_container psql -U postgres

# Crear base de datos
CREATE DATABASE universidad;
\c universidad

# Ejecutar scripts
\i /ruta/a/BDEjemploUni_Create.sql
\i /ruta/a/BDEjemploUni_Insert.sql
```

Opción C: Copiar scripts al contenedor

```
# Copiar archivos SQL
docker cp L0/SQL/ postgres_container:/tmp/

# Ejecutar dentro del contenedor
docker exec -it postgres_container bash
psql -U postgres -d universidad -f /tmp/SQL/BDEjemploUni_Create.sql
```

Paso 3: Práctica con Consultas SQL (L1 y L2)

```
# L1 - Consultas básicas
psql -U postgres -d bpsimple -f L1/bpsimple.sql

# L2 - Consultas avanzadas
psql -U postgres -d universidad -f L2/universidad.sql
```

Recomendación: Ejecutar consultas una por una en pgAdmin para entender cada resultado.

Paso 4: Programación con Python (L3)

```
cd L3

# 1. Crear entorno virtual (recomendado)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 2. Instalar dependencias
```

```
pip install psycopg[binary] sqlalchemy pandas asyncpg matplotlib

# 3. Configurar credenciales
# Editar myconfig.ini con los datos de conexión

# 4. Ejecutar scripts
python universidad_01_connection.py
python pandas_read_sql.py
python asyncpg_select.py
```

Paso 5: Base de Datos DVDRental

La base de datos DVDRental se carga desde un archivo `.tar` (no incluido en esta carpeta pero disponible en PostgreSQL.org):

```
# 1. Descargar dvdrental.tar desde:
# https://www.postgresqltutorial.com/postgresql-getting-started/postgresql-sample-database/

# 2. Crear base de datos
docker exec -it postgres_container psql -U postgres -c "CREATE DATABASE dvdrental;"

# 3. Restaurar desde backup
docker cp dvdrental.tar postgres_container:/tmp/
docker exec -it postgres_container pg_restore -U postgres -d dvdrental /tmp/dvdrental.tar

# 4. Verificar
docker exec -it postgres_container psql -U postgres -d dvdrental -c "\dt"
```

Paso 6: MongoDB (Opcional)

```
cd L3/mongodb
docker-compose up -d

# Acceder a Mongo Express
# Navegador: http://localhost:8081
```

Notas Técnicas

PostgreSQL vs MySQL

Este material utiliza PostgreSQL porque:

- Mayor conformidad con estándares SQL
- Funcionalidades avanzadas (window functions, CTEs)
- Tipos de datos ricos (JSON, arrays, rangos)
- Excelente rendimiento en consultas complejas

Docker vs Instalación Nativa

Se utiliza Docker porque:

- Entorno reproducible en cualquier sistema operativo
- Aislamiento de versiones y configuraciones
- Fácil limpieza y reinicio del entorno
- Incluye pgAdmin preconfigurado

Librerías Python

- **psycopg3**: Driver oficial, recomendado para producción
- **SQLAlchemy**: ORM potente, abstracción de base de datos
- **asyncpg**: Máximo rendimiento, programación asíncrona
- **pandas**: Análisis de datos, integración con NumPy y matplotlib

---

Seguridad y Mejores Prácticas

Para Producción (NO usar en este entorno de aprendizaje)

ADVERTENCIA: Las contraseñas de ejemplo son para fines educativos. En producción: - Usar variables de entorno para credenciales - Implementar certificados SSL/TLS - Configurar firewalls y redes privadas - Utilizar usuarios con privilegios mínimos - Implementar backups automáticos

Para el Laboratorio

Las configuraciones están optimizadas para aprendizaje: - Contraseñas simples para facilitar el acceso - Puertos expuestos para conectividad local - Sin autenticación en Mongo Express para simplicidad

---

Solución de Problemas Comunes

PostgreSQL no inicia

```
# Verificar logs
docker logs postgres_container

# Verificar que el puerto 5432 está libre
netstat -an | grep 5432

# Reiniciar contenedor
docker-compose restart
```

Error de conexión desde Python

```python
# Verificar conectividad básica
import psycopg
conn = psycopg.connect(
    "host=localhost port=5432 dbname=universidad user=postgres password=changeme"
)
print("Conexión exitosa")
```

pgAdmin no carga

```bash
# Verificar que el contenedor está corriendo
docker ps | grep pgadmin

# Limpiar volúmenes y reiniciar
docker-compose down -v
docker-compose up -d
```

Licencia

Material docente en abierto de la Universidad Rey Juan Carlos.
Algunos derechos reservados. Este documento se distribuye bajo "Atribución-CompartirIgual 4.0 Internacional" de Creative Commons.

Autores:

- David Moreno Lumbreras
- Daniela Patricia Feversani

Todos los derechos reservados (c) 2025-2026 URJC.

Última actualización: Octubre 2025
Versión: 1.0
Compatible con: PostgreSQL 16, Python 3.8+